

# 第六题：MNIST 分类中 SGD 与自然梯度训练的对比

2026 年 7 月 5 日

## 1 题目与实现说明

第六题要求构建神经网络对 MNIST 数据集中的 0-9 手写数字进行分类，并分别使用随机梯度下降（SGD）和自然梯度（Natural Gradient, NG）训练，最后比较训练结果。

本解答的程序为 `mnist_sgd_ng.py`。程序优先读取标准 MNIST IDX 文件，包括训练图像、训练标签、测试图像和测试标签四个文件；文件名可带 `.gz` 后缀。若这些文件放在 `data/mnist` 中，可直接运行：

```
python mnist_sgd_ng.py --mnist-dir data/mnist --require-mnist.
```

当前已经将标准 MNIST IDX 文件下载到 `data/mnist`，因此本节报告采用真实 MNIST 全量训练集和测试集进行增强实验。为了提高两种方法的最终准确率，相比快速测试版本，本次增加了训练样本数、测试样本数、训练轮数和隐层宽度，并适当降低 SGD 学习率，使训练过程更稳定。

## 2 神经网络模型

采用一个单隐层多层感知机（MLP）：

$$h = \text{ReLU}(xW_1 + b_1), \quad p_\theta(y | x) = \text{softmax}(hW_2 + b_2),$$

其中输入  $x \in \mathbb{R}^{784}$ ，隐层维度为 128，输出层为 10 类。损失函数为带  $L_2$  正则项的交叉熵：

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^n \log p_\theta(y_i | x_i) + \frac{\lambda}{2} (\|W_1\|_2^2 + \|W_2\|_2^2).$$

## 3 SGD 与自然梯度方法

SGD 直接沿小批量梯度的反方向更新参数：

$$\theta_{t+1} = \theta_t - \eta \hat{g}_t,$$

其中  $\hat{g}_t = \nabla_\theta \mathcal{L}_B(\theta_t)$  为当前 minibatch 的梯度。

自然梯度的思想是不用欧氏距离衡量参数变化，而用模型分布诱导出的 Fisher 信息矩阵  $F(\theta)$  衡量参数空间的局部几何：

$$\theta_{t+1} = \theta_t - \alpha F(\theta_t)^{-1} \hat{g}_t.$$

对 MLP 精确构造和求逆完整 Fisher 矩阵代价很高，因此程序采用对角经验 Fisher 近似：

$$\hat{F}_t = \beta \hat{F}_{t-1} + (1 - \beta)(\hat{g}_t \odot \hat{g}_t),$$

$$\theta_{t+1} = \theta_t - \alpha \left( \text{diag}(\hat{F}_t) + \lambda_{\text{NG}} I \right)^{-1} \hat{g}_t.$$

其中  $\lambda_{\text{NG}}$  为阻尼项，用于避免 Fisher 对角元素过小导致步长过大。这种实现保留了自然梯度按参数敏感度自适应缩放梯度的核心思想，同时避免了完整 Fisher 矩阵的存储和求逆。

## 4 实验设置

两种优化器使用相同的数据划分、相同初始参数和相同 minibatch 顺序。主要参数如下：

| 项目             | 数值    |
|----------------|-------|
| 训练集大小          | 60000 |
| 测试集大小          | 10000 |
| 隐层神经元          | 128   |
| 训练轮数           | 20    |
| Batch size     | 128   |
| SGD 学习率        | 0.08  |
| NG 学习率         | 0.03  |
| NG 阻尼          | 0.5   |
| NG Fisher 衰减系数 | 0.9   |



图 1: 标准 MNIST 数据集中的  $28 \times 28$  手写数字样例

图 1 展示了本次真实 MNIST 实验中抽取的数字样例。每个样本是  $28 \times 28$  灰度图，数字存在明显的书写风格、粗细、倾斜和局部形变差异，因此比前期离线模板数据更接近真实分类任务。

## 5 运行结果对比

本次运行的最终结果如下：

| 优化器 | 最终训练准确率 | 最终测试准确率 | 最终测试损失 | 最佳测试准确率 |
|-----|---------|---------|--------|---------|
| SGD | 0.9845  | 0.9741  | 0.1072 | 0.9741  |
| NG  | 0.9797  | 0.9717  | 0.1169 | 0.9717  |

图 2 给出了测试集准确率随训练轮数变化的曲线。红色曲线为 SGD，蓝色曲线为 NG。可以看到两种方法在真实 MNIST 上都随训练轮数稳定提升。SGD 第 10 轮达到 0.9650，第 20 轮达到 0.9741；NG 第 10 轮达到 0.9600，第 20 轮达到 0.9717。与快速测试版本相比，使用全量训练集、扩大隐层并增加训练轮数后，两种方法的最终准确率都显著提高。SGD 仍略高于对角 Fisher 近似 NG，但二者差距已经较小。

图 3 和图 4 为两种优化器在 10000 个 MNIST 测试样本上的混淆矩阵。矩阵的行表示真实类别，列表示预测类别；对角线越亮、非对角线数字越少，说明分类越准确。两种方法的大多数错误仍集中在形状相近的类别之间，例如 4 与 9、3 与 5、7 与 9、8 与 9 等。SGD 的测试准确率略高，

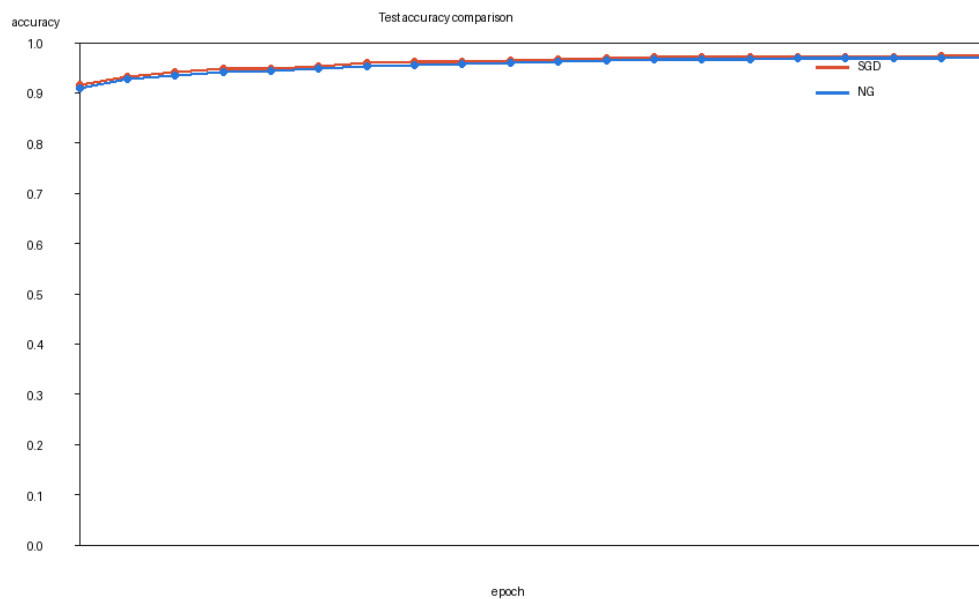


图 2: SGD 与 NG 的测试准确率曲线

SGD confusion matrix

|   | 0   | 1    | 2    | 3   | 4   | 5   | 6   | 7   | 8   | 9   |
|---|-----|------|------|-----|-----|-----|-----|-----|-----|-----|
| 0 | 966 |      | 2    | 2   | 2   | 2   | 1   | 1   | 2   | 2   |
| 1 |     | 1121 | 2    | 2   |     |     | 4   | 1   | 5   |     |
| 2 | 4   | 2    | 1002 | 7   | 2   |     | 2   | 7   | 6   |     |
| 3 |     |      | 3    | 996 |     | 1   |     | 3   | 6   | 1   |
| 4 |     |      | 7    |     | 961 |     | 4   | 2   | 1   | 7   |
| 5 | 3   | 1    |      | 14  | 1   | 856 | 7   | 1   | 9   |     |
| 6 | 4   | 3    |      |     | 6   | 7   | 934 |     | 4   |     |
| 7 | 1   | 7    | 11   | 5   |     |     |     | 998 | 1   | 5   |
| 8 | 3   | 2    | 1    | 8   | 4   | 4   | 4   | 4   | 944 |     |
| 9 | 3   | 6    |      | 12  | 11  | 4   | 1   | 5   | 4   | 963 |

predicted

图 3: SGD 混淆矩阵

NG confusion matrix

|   | 0   | 1    | 2    | 3   | 4   | 5   | 6   | 7   | 8   | 9   |
|---|-----|------|------|-----|-----|-----|-----|-----|-----|-----|
| 0 | 966 |      | 1    | 2   | 1   | 3   | 3   | 1   | 1   | 2   |
| 1 |     | 1121 | 2    | 2   |     |     | 4   | 1   | 5   |     |
| 2 | 4   | 2    | 1006 | 6   | 2   |     | 1   | 6   | 5   |     |
| 3 |     |      | 4    | 990 |     | 2   |     | 6   | 7   | 1   |
| 4 | 2   |      | 8    |     | 955 |     | 3   | 2   | 2   | 10  |
| 5 | 4   | 1    |      | 11  | 2   | 853 | 9   | 1   | 8   | 3   |
| 6 | 6   | 3    |      |     | 6   | 8   | 931 |     | 4   |     |
| 7 | 1   | 7    | 12   | 6   |     |     |     | 992 | 1   | 9   |
| 8 | 3   | 3    | 1    | 8   | 4   | 4   | 4   | 5   | 942 |     |
| 9 | 4   | 6    |      | 10  | 14  | 5   | 1   | 5   | 3   | 961 |

predicted

图 4: NG 混淆矩阵

说明在当前单隐层网络和超参数下，普通 SGD 对 MNIST 的拟合更充分；NG 的混淆矩阵整体结构与 SGD 相似，说明对角 Fisher 近似同样学到了主要类别边界。

## 6 解释与讨论

从曲线可以看出，增加训练数据和训练轮数是提升准确率最直接的手段。此前快速测试只使用 4000 个训练样本和 12 轮训练，SGD 与 NG 的测试准确率分别约为 0.9070 和 0.8820；本次使用全部 60000 个训练样本、20 轮训练和 128 个隐层神经元后，二者分别提升到 0.9741 和 0.9717。

这个结果有三点原因。第一，真实 MNIST 的类别内变化较大，更多训练样本能显著改善模型对不同书写风格的覆盖。第二，隐层从 64 增加到 128 后，网络有更强的非线性表示能力。第三，将 SGD 学习率从快速测试中的较大值降低到 0.08 后，训练曲线更平滑，后期不易出现明显振荡。对 NG 而言，对角 Fisher 近似能提供一定的参数尺度自适应，但它忽略了参数间相关性，因此最终仍略低于 SGD。若进一步使用卷积神经网络、数据增强或 K-FAC 等更强的自然梯度近似，准确率还有继续提升空间。

## 7 文件说明

- `mnist_sgd_ng.py`: 主程序，包含数据读取、MLP、SGD/NG 训练和结果输出；
- `requirements.txt`: 运行依赖；
- `data/mnist/`: 标准 MNIST IDX 数据文件；
- `results_mnist_full/results.csv`: 每轮训练与测试指标；
- `results_mnist_full/summary.json`: 最终对比摘要；
- `results_mnist_full/accuracy_curve.png`: 准确率曲线；
- `results_mnist_full/confusion_sgd.png` 与 `results_mnist_full/confusion_ng.png`: 混淆矩阵。